
Sound Analysis: Emergency Vehicle Sirens

Sophia Huang, Peter
Pavlatos, Lydia Wyble





01

The Algorithm



CNN: Purpose

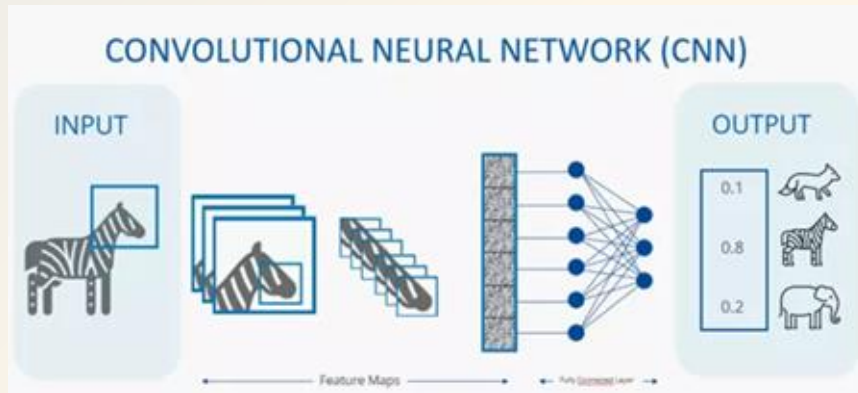
Convolutional Neural Networks are deep learning networks optimized for classifying grid data (e.g. image files)

- **Goal:** Pass dense, complex data through a series of hidden layers and transformations

Our Problem: Audio File Classification

Input: Feature vectors selected via Mel-Frequency Cepstral Coefficients (MFCC)

Output: Class (ambulance, firetruck, traffic)





CNN: Basic Idea

- Inputs are passed through hidden layers to produce a singular output
 - Convolutional and pooling layers to compress dimensions
 - Flattening to produce final classification

Sequential (Input shape: 256 x 80)			
Layer (type)	Output Shape	Param #	Trainable
256 x 1 x 80			
Unflatten			
256 x 16 x 80			
Conv1d		64	True
BatchNorm1d		32	True
ReLU			
256 x 16 x 40			
MaxPool1d			
256 x 32 x 40			
Conv1d		1568	True
BatchNorm1d		64	True
ReLU			
256 x 32 x 20			
MaxPool1d			
256 x 64 x 20			
Conv1d		6208	True
BatchNorm1d		128	True
ReLU			
256 x 64 x 10			
MaxPool1d			
256 x 64 x 1			
AdaptiveAvgPool1d			
256 x 64			
Flatten			
256 x 3			
Linear		195	True
Total params: 8,259			
Total trainable params: 8,259			
Total non-trainable params: 0			



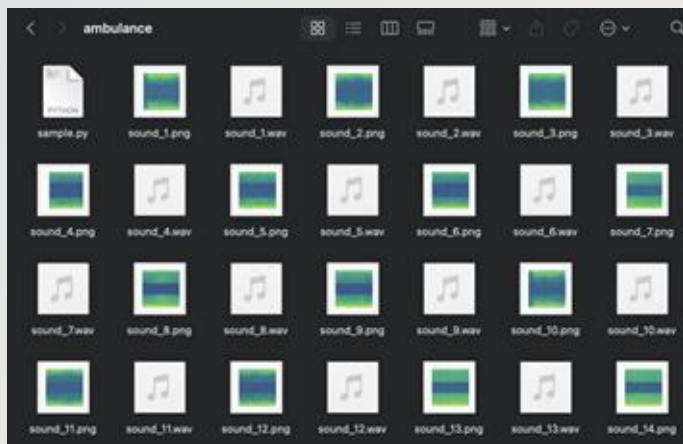
02

Experimental Plan

Dataset

Emergency Vehicle Siren Sounds by Vishnu U. on Kaggle

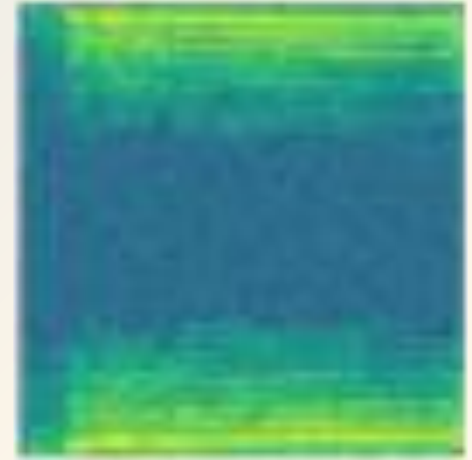
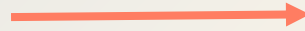
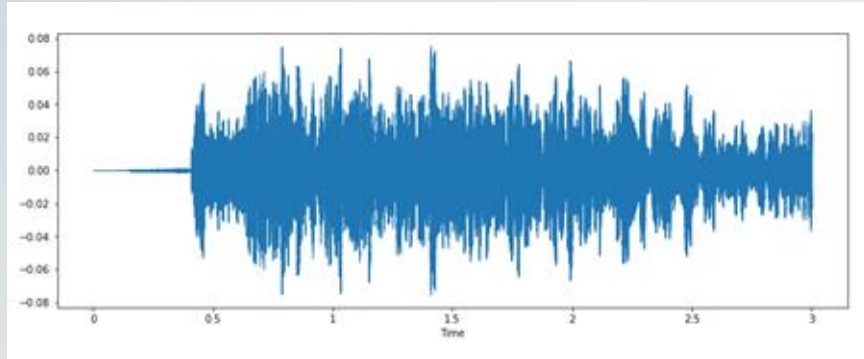
- 600 observations
 - Three-second .wav files scraped from YouTube
 - .png file with the spectrogram image of each audio file
- Ambulance, firetruck, and plain traffic noises
- **Application:** Sound recognition for assistive hearing technology



Sample of Data



Spectrogram Creation



Axes represent time and frequency
Color represents amplitude





03

Demo of CNN

Imported Libraries



```
# Imports and Libraries
import numpy as np
import pandas as pd

# Audio Processing Libraries
## https://medium.com/@rijuldahiya/a-comprehensive-guide-to-audio-processing-with-librosa-in-python-a49276387a4b
import librosa
import librosa.display
from scipy import signal

# Import resampy (for an error that comes up later)
import sys
!{sys.executable} -m pip install resampy

# Deal with .pkl files
## https://docs.python.org/3/library/pickle.html
import pickle

# CNN
import torch
from fastai.data.block import DataBlock, TransformBlock, CategoryBlock
from fastai.data.transforms import RandomSplitter
from fastai.vision.all import *
import torch.nn as nn
import torch.nn.functional as F

Collecting resampy
  Downloading resampy-0.4.3-py3-none-any.whl.metadata (3.0 kB)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from resampy) (2.0.2)
Requirement already satisfied: numba>=0.53 in /usr/local/lib/python3.12/dist-packages (from resampy) (0.60.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.53->resampy) (0.43.0)
Downloading resampy-0.4.3-py3-none-any.whl (3.1 MB)
3.1/3.1 MB 33.3 MB/s eta 0:00:00
Installing collected packages: resampy
Successfully installed resampy-0.4.3
```



Import Kaggle Dataset



```
%env KAGGLE_USERNAME=sophia4827
%env KAGGLE_KEY=ac334c3351a2af38b3f6b0ce9d9922a6

!kaggle datasets download vishnu0399/emergency-vehicle-siren-sounds
!unzip emergency-vehicle-siren-sounds.zip

... inflating: sounds/traffic/sound_572.png
    inflating: sounds/traffic/sound_572.wav
    inflating: sounds/traffic/sound_573.png
    inflating: sounds/traffic/sound_573.wav
    inflating: sounds/traffic/sound_574.png
    inflating: sounds/traffic/sound_574.wav
    inflating: sounds/traffic/sound_575.png
    inflating: sounds/traffic/sound_575.wav
    inflating: sounds/traffic/sound_576.png
    inflating: sounds/traffic/sound_576.wav
    inflating: sounds/traffic/sound_577.png
    inflating: sounds/traffic/sound_577.wav
    inflating: sounds/traffic/sound_578.png
    inflating: sounds/traffic/sound_578.wav
```



Preprocessing



```
# Mel-Frequency Cepstral Coefficients (MFCCs)
# MFCCs because it is common speech recognition tool
## essentially a way to represent the shape of a sound
## https://librosa.org/doc/latest/tutorial.html

def features_extractor(file_name):
    audio, sample_rate = librosa.load(file_name, res_type='kaiser_fast') # Loads the audio
    mfccs_features = librosa.feature.mfcc(y=audio, sr=sample_rate, n_mfcc=80) # Computes MFCCs
    mfccs_scaled_features = np.mean(mfccs_features.T, axis=0) # Averages them over time

    return mfccs_scaled_features # Returns an 80-dimensional NumPy array

audio_dataset_path = '/content/sounds/'

extracted_features = []
for path in os.listdir(audio_dataset_path): # Loops through each folder
    for file in os.listdir(audio_dataset_path+path+"/"): # Loops through all the files in the folder
        if file.lower().endswith(".wav"): # Looking for the ones that end in .wav (the audio file)
            file_name = audio_dataset_path+path+"/"+file # Full path
            data = features_extractor(file_name)
            extracted_features.append([data, path]) # data = feature, path = class
```





Our Dataframe

```
sound_df = pd.DataFrame(extracted_features, columns=['feature', 'class'])
sound_df.head()
```

	feature	class
0	[0.4927528, 70.27676, -53.856594, 9.528141, -25.403837, 15.2287, -15.1158495, 19.300373, -14.393512, 19.427523, -8.059544, 18.845385, -2.7397153, 9.550518, -15.190494, 2.8947866, -16.018461, 10.162927, -15.398781, 0.28483585, -16.732883, 2.9447973, -5.09737, 8.18878, -5.7554603, 1.875593, -8.225499, 1.9528755, -3.5899963, 7.84778, 4.3428284, 9.476964, 0.3902035, -1.2717514, -11.386358, 0.21098909, 1.7850833, 0.079594, 0.7772239, -4.631893, -11.28215, -4.4981594, -3.2897906, 0.7177108, -4.1935034, -2.0153413, 0.37900567, 1.9824747, -1.8371483, -4.0852265, -5.2385397, 0.5231786, 0.2214727, ...]	firetruck
1	[-14.551583, 100.671364, -67.20877, 22.368254, -43.999752, 10.656855, -17.30652, 5.362701, -8.741289, 3.3850975, -14.356076, 9.697705, -12.762473, 9.403711, -14.274545, 0.34238955, -15.5461235, 1.6313418, -8.663719, 8.4869175, -4.7459865, 6.622468, -7.6283135, 0.9972842, -11.4253235, -3.256607, -8.459014, 0.2374873, -5.8098617, 0.3395532, -3.3490272, 5.333056, 0.5413731, 2.792037, -0.8011543, 3.7590237, 1.8342698, 1.7706069, -4.1412325, -2.103218, -0.9550036, 2.4895358, 0.48883265, -0.20314455, -5.127532, -4.7529693, -3.0726464, -2.899061, -3.6919992, -2.0477045, 0.5074704, 1.1609316, 0.33...	firetruck
2	[-90.88731, 86.63253, -70.92562, 6.459669, -43.16606, 7.637469, -28.587132, 3.454756, -8.281891, 12.820457, 3.1870372, 10.389404, -11.986944, -5.5205183, -15.774358, 5.311144, -7.948529, 2.8761878, -3.128715, 12.33212, -3.7772548, -5.2047405, -11.48532, -3.2232013, -9.594206, -3.8694592, -5.384892, 0.07300063, -1.1487675, 4.2829723, 4.537278, 5.4793305, 2.7150097, 3.904271, 2.898826, 3.4837968, -1.7181859, -4.0284038, -0.15126179, 4.6906233, 1.5226752, -1.6662475, -4.44736, -7.829282, -2.0327904, -0.8344852, -0.7474671, -1.2721409, -1.8606224, -0.20892404, -1.5400206, -3.4352686, -2.225042...	firetruck
3	[-102.93468, 86.28514, -53.082405, 4.2343984, -10.076095, 16.043158, -18.077303, 28.747427, -12.934573, 6.8472447, -4.657748, 0.84946644, -15.261891, -3.825292, -21.548294, 4.36271, -14.92331, 17.013615, -3.7159226, -0.8884975, -11.979718, -2.4143202, -15.94286, -1.7183058, -19.343344, 2.291455, 1.3093864, 2.9262898, -7.871549, 0.84772986, -2.2928152, 4.594571, 0.9898489, 13.415411, 6.4008164, 1.721959, -0.5905902, -2.6300387, -6.526681, -2.4775715, -6.5337796, -5.415266, -4.966041, -0.044085898, -4.875382, -1.5850703, -1.4351468, -3.5226517, -4.5176315, -2.3467088, -3.6587582, -1.0829514, ...]	firetruck
4	[-147.95042, 100.03297, -49.093147, -3.2169523, -16.16093, 24.553368, 1.3413548, 24.288229, -7.4364934, 12.342838, -8.475367, 3.452299, -8.801905, 17.549885, 8.637344, 12.024434, -10.657864, -5.8812637, -14.07014, 3.3203309, -6.437163, -1.41544, -5.7053957, -1.5362384, -2.6174417, 4.0036464, 2.3587604, 3.0183349, -3.120856, -3.863868, 6.8793267, 13.159447, 7.3666835, 5.0851784, -1.1115203, -6.019041, -5.6717498, 2.9888728, -0.11752061, -9.470477, 7.667115, 1.0880189, 0.80747426, -0.40723225, -0.97826856, -5.071155, -3.2818565, -2.2063138, -4.581456, -7.9406805, 0.85438305, 6.0728974, -0.4...	firetruck



DataBlock & DataLoaders



```
X = np.stack(sound_df['feature'].to_list()).astype('float32') # (n_samples, n_features)
y = sound_df['class'].to_list() # List of string labels
idxs = np.arange(len(X))

# Convert to a PyTorch float tensor
MFCCBlock = TransformBlock(type_tfms = lambda o: torch.tensor(o, dtype=torch.float32))

# Takes an index and returns the feature vector
def get_x(i): return X[i] # Numpy array
def get_y(i): return y[i] # String label

# Turns data into something a model can train on
sound = DataBlock(
    blocks=(MFCCBlock, CategoryBlock),
    get_items=lambda _: idxs, # Items are indices 0..N-1
    get_x=get_x,
    get_y=get_y,
    splitter=RandomSplitter(seed=42) # Splits training and validation
)

# Feeds data to your model in batches during training
sound_dls = sound.dataloaders(source=idxs, bs=256)
```



Our CNN

```
n_classes = sound_dls.c # number of labels

cnn = nn.Sequential(
    nn.Unflatten(1, (1, 80)),      # [bs, 80] -> [bs, 1, 80]
    nn.Conv1d(1, 16, kernel_size=3, padding=1),
    nn.BatchNorm1d(16),
    nn.ReLU(),
    nn.MaxPool1d(2),               # [bs, 16, 40]

    nn.Conv1d(16, 32, kernel_size=3, padding=1),
    nn.BatchNorm1d(32),
    nn.ReLU(),
    nn.MaxPool1d(2),               # [bs, 32, 20]

    nn.Conv1d(32, 64, kernel_size=3, padding=1),
    nn.BatchNorm1d(64),
    nn.ReLU(),
    nn.MaxPool1d(2),               # [bs, 64, 10]

    nn.AdaptiveAvgPool1d(1),       # [bs, 64, 1]
    nn.Flatten(),                  # [bs, 64]
    nn.Linear(64, n_classes)      # [bs, n_classes]
)
```




04

Results of CNN



Running the Model



```
# learner
learn = Learner(sound_dls, cnn, loss_func=CrossEntropyLossFlat(), metrics=accuracy)

# training
learn.fit_one_cycle(100, 1e-3)
```

First 10
epoch

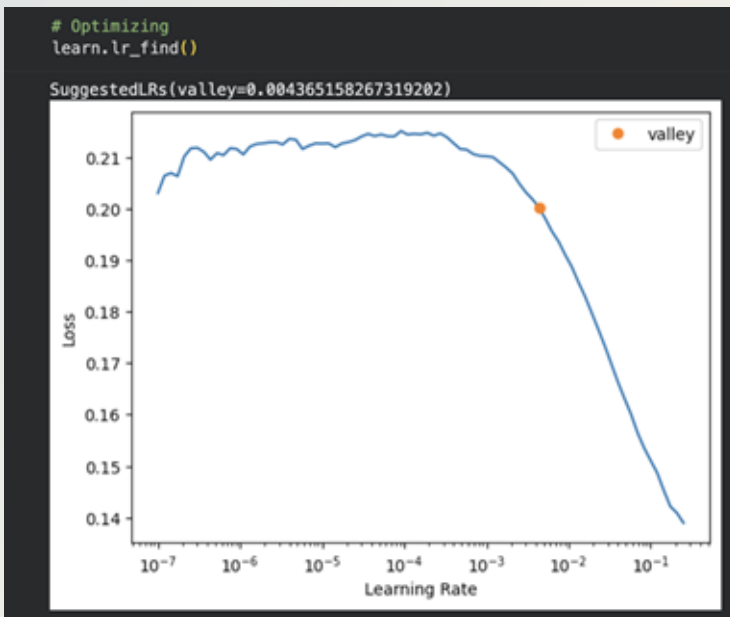
epoch	train_loss	valid_loss	accuracy	time
0	1.111061	1.123018	0.258333	00:00
1	1.112473	1.112102	0.258333	00:00
2	1.114390	1.106296	0.483333	00:00
3	1.109801	1.101925	0.483333	00:00
4	1.108382	1.099028	0.458333	00:00
5	1.106317	1.095994	0.408333	00:00
6	1.101718	1.092618	0.408333	00:00
7	1.098073	1.088543	0.408333	00:00
8	1.093212	1.083708	0.433333	00:00
9	1.088332	1.077923	0.450000	00:00

Last 10
epoch

epoch	train_loss	valid_loss	accuracy	time
90	0.377542	0.267256	0.933333	00:00
91	0.373912	0.267119	0.933333	00:00
92	0.369538	0.266639	0.933333	00:00
93	0.365988	0.266761	0.933333	00:00
94	0.362213	0.266043	0.933333	00:00
95	0.358495	0.265342	0.933333	00:00
96	0.354953	0.264449	0.933333	00:00
97	0.351461	0.263513	0.933333	00:00
98	0.348487	0.263391	0.933333	00:00
99	0.345151	0.263213	0.933333	00:01



Learning Rate Finder



Fine Tuning the Model



```
# Fine tune learner with better learning rate  
learn.fine_tune(100, base_lr=0.004)
```

First 10
epoch

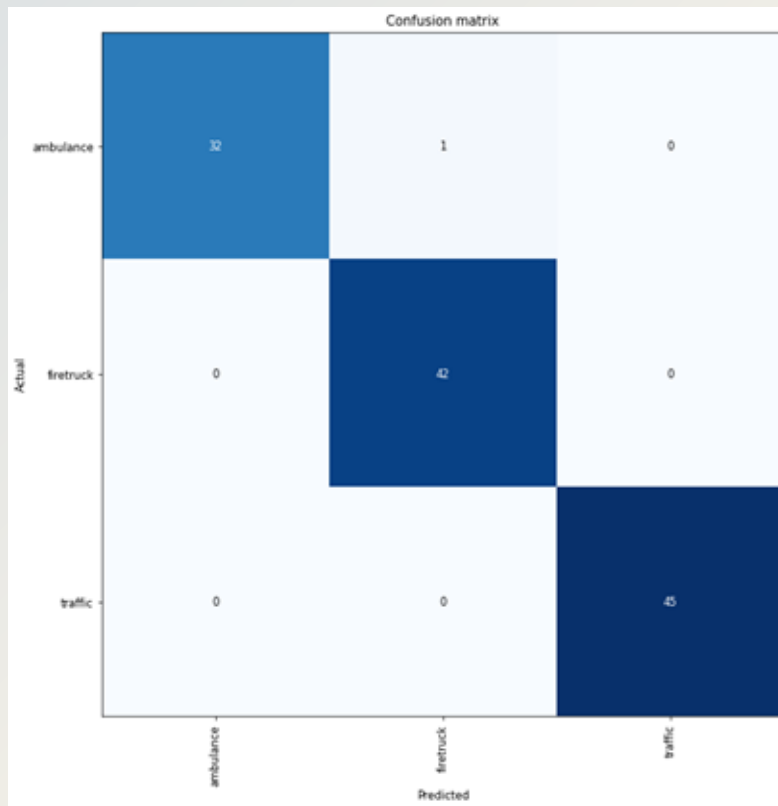
epoch	train_loss	valid_loss	accuracy	time
0	0.103363	0.150535	0.966667	00:00
1	0.095798	0.146560	0.966667	00:00
2	0.092166	0.142425	0.958333	00:00
3	0.093896	0.140465	0.958333	00:00
4	0.092890	0.138745	0.958333	00:00
5	0.091671	0.137396	0.958333	00:00
6	0.092045	0.136898	0.958333	00:00
7	0.092146	0.136105	0.958333	00:00
8	0.091598	0.136567	0.958333	00:00
9	0.090519	0.135556	0.958333	00:00

Last 10
epoch

epoch	train_loss	valid_loss	accuracy	time
90	0.026623	0.048813	0.991667	00:00
91	0.026262	0.048722	0.991667	00:00
92	0.025891	0.048492	0.991667	00:00
93	0.025540	0.048173	0.991667	00:00
94	0.025218	0.048265	0.991667	00:00
95	0.024870	0.048154	0.991667	00:00
96	0.024541	0.048098	0.991667	00:00
97	0.024239	0.048106	0.991667	00:00
98	0.023963	0.047863	0.991667	00:00
99	0.023634	0.047886	0.991667	00:00



Confusion Matrix



```
# Confusion matrix  
interp = ClassificationInterpretation.from_learner(learn)  
interp.plot_confusion_matrix(figsize=(10,10), dpi=60)
```





05

Alternative Model



SVM (Support Vector Machine)

- **Idea:** SVM may be similarly accurate with a lower computation time
- **Methodology:** Used scikit-learn to train a polynomial SVM model with $C = 1$ (optimal margin chosen by GridSearchCV). Note: This model used the image (spectrogram) representations of the data.
- **Result:** Model trains quickly (~45 seconds), but is less accurate than CNN. For our proposed application, improving accuracy is worth the increased computation time.
- **Accuracy:** 0.95
 - All misclassifications falsely assigned ambulance to firetruck (0, 2) or vice versa.



Truth	0	1	2
Predicted			
0	57	0	6
1	0	60	0
2	3	0	54



06

Limitations & Future Work



Limitations & Future Work

Largest limitation in the dataset is **size**:

- Only 600 observations
- Data in each category is audibly “distinct”
 - In the real world, there are overlapping sounds and sometimes multiple categories present at once

Future Work: Large-scale data collection, including samples where there is heavy background noise or overlapping sirens

- Would likely reduce model accuracy or require a more complex classification algorithm





Concluding Remarks

Algorithm: Convolutional Neural Network (CNN)

- Uses hidden layers to extract features from data and predict classes

Application: Train a CNN on vectorized audio data from traffic noise and emergency vehicle sirens

- Goal: Improved sound recognition for assisted hearing technology

Results: Model predicted new observations with 0.99 accuracy

- Impact of results is limited by small dataset

Next Steps: Repeat experiment with a larger, more diverse dataset





Thank You!

Questions?





References

Dataset: <https://www.kaggle.com/datasets/vishnu0399/emergency-vehicle-siren-sounds/code>

<https://www.ionos.com/digitalguide/websites/web-development/convolutional-neural-networks/>

